

Conteneurs et Docker

Comprendre l'isolation des processus, les conteneurs et leur orchestration dans les infrastructures modernes.



**À quoi a accès un processus sur un
système d'exploitation ?**



Sommaire

De quoi s'agit-il ?

01

Introduction

02

Le point de vue développeur

03

Le point de vue infrastructure

04

Les solutions

05

Docker

06

Docker en pratique



Introduction

*Le point de vue
développeur*

*Le point de vue
infrastructure*

Les solutions

Docker

Introduction



L'idée générale

Les infrastructures modernes doivent déployer et maintenir des applications

Ces applications reposent sur :

- Des processus
- Des dépendances
- Des configurations spécifiques

La **conteneurisation** est une réponse système aux problématiques de déploiement applicatif.

Un moment de réflexion

Sur un OS, un processus accède à quoi ?

- Des fichiers
- Des bibliothèques
- Des interfaces réseau
- Des ressources CPU et mémoire

Par défaut, un processus voit plus que ce qui lui est strictement nécessaire. Cela pose des problèmes de :

- Sécurité
- Stabilité
- Isolation



Processus et FS

Le Système de Fichiers (FS) d'un OS contient :

- Des fichiers système
- Des bibliothèques partagées
- Des données applicatives

Les mécanismes classiques de permissions sont nécessaires mais souvent insuffisants pour cloisonner finement.

=> En adminsyst on doit réduire la surface d'attaque.



Une solution : le cloisonnement

Une vieille méthode :

- Commande **chroot** sous Linux
- Commande **jails** sous BSD

L'objectif est de :

- Limiter ce qu'un processus peut voir
- Limiter l'impact d'un incident (brèche)

=> Bases de la conteneurisation.



Le conteneur

Un **conteneur** est un ensemble de processus s'exécutant dans un environnement isolé.

- Il dispose de son propre FS
- Il dispose de ressources limitées
- Il s'appuie sur le noyau du système hôte



La conteneurisation

L'idée générale :

- Cloisonner un processus (et ses processus fils)
- Déclarer explicitement les ressources accessibles et l'environnement d'exécution
- Créer des instances séparées reproductibles et indépendantes

=> Extension moderne du concept de prison logicielle



Conteneur ou VM ?

Même si à l'utilisation les conteneurs ressemblent à des machines virtuelles (VM), c'est en fait très différents.

Pour avoir une VM, un hyperviseur propose une couche d'abstraction qui reproduit le comportement d'une machine réelle et permet ainsi l'installation d'un OS complet.



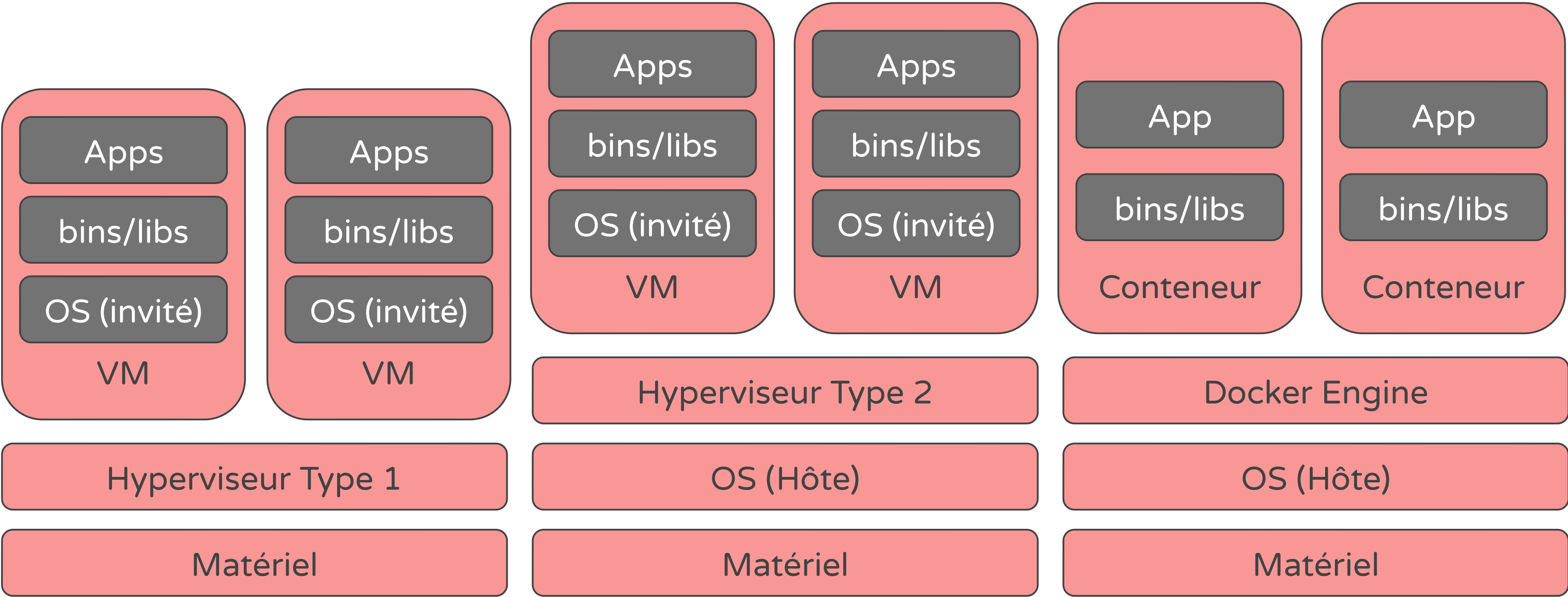
Conteneur ou VM ? (suite)

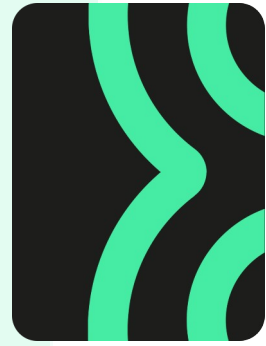
Avec les conteneurs, il y a un seul noyau qui tourne et c'est en s'appuyant sur des fonctionnalités de ce noyau ([namespaces](#), [Cgroups](#)...) qu'on isole des processus dans des conteneurs.

C'est donc moins consommateur en ressources (CPU, mémoire et disque).



Schéma des différences





Introduction

*Le point de vue
développeur*

*Le point de vue
infrastructure*

Les solutions

Docker

Docker en pratique

Le point de vue développeur



Problème initial

Pour qu'une application fonctionne, elle a besoin de versions de langages (Node, Python, Java...), de bibliothèques, de variables d'environnement, etc.

L'environnement d'exécution peut être différent :

- Machine de test
- Poste de travail du développeur
- Serveur de test
- Serveur de production, etc.



Diversité des environnements

- Dev, test, pré-prod, prod
- Linux, Windows, Mac
- Version de bibliothèques et de dépendances
- Configurations réseau différentes
- Permissions différentes

=> Test et debug compliqué et difficile à mettre en oeuvre

=> Déploiement à risque



La phrase que vous pourrez entendre !

Ça marche pas sur ton PC ?
C'est bizarre ça marche sur ma
machine...

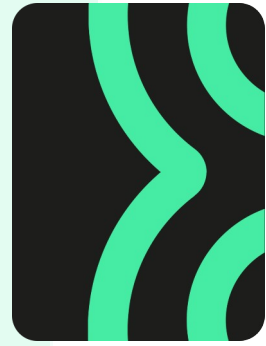


Ce qu'apporte la conteneurisation aux dev

L'application est livrée avec :

- Ses propres dépendances
- Sa propre configuration

Elle a le même comportement partout, quel que soit la machine ou l'environnement.



Introduction

*Le point de vue
développeur*

*Le point de vue
infrastructure*

Les solutions

Docker

Docker en pratique

Le point de vue infrastructure



Problème initial

L'infrastructure doit héberger plusieurs applications (ou services) avec les impératifs suivants :

- Hébergement multiple
- Garanti de sécurité
- Garanti de l'accès aux ressources
- Optimisation de l'utilisation des ressources



Sans les conteneurs

- Services installés (liés) sur OS spécifique
- Dépendances partagées entre les services
- Conflits de versions (OS/services)
- Difficulté d'isolation d'un service compromis



Rappel : rôle et missions d'un TSSR

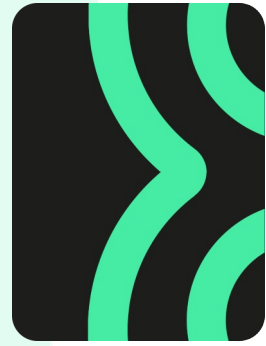
- Réduire la surface d'attaque
- Gérer les ressources (CPU, RAM, espace disque)
- Déploiement fiable et reproductible
- MAJ sans problème



Ce qu'apporte la conteneurisation aux gestionnaires d'infra

- Cloisonnement strict des processus
- Déploiements/clonage plus rapides
- Indépendant de l'OS
- Pas de trace sur les disques

=> Bonne alternative aux VM



Introduction

*Le point de vue
développeur*

*Le point de vue
infrastructure*

Les solutions

Docker

Docker en pratique

Les solutions



Docker mais pas que !

La conteneurisation est un concept.
Docker est une des implémentations de ce concept.

- Les approches possibles de ce concept :
- La conteneurisation bas niveau
 - La conteneurisation système
 - La conteneurisation applicative
 - L'orchestration de conteneurs (avancé)



La conteneurisation bas niveau

On peut cloisonner des ressources avec quelques commandes :

- chroot
- namespaces & cgroups
- jails



La conteneurisation système

LXC (*Linux Containers*) permet de créer des conteneurs système “ressemblant” à une VM.

On dispose alors d’une machine Linux complète
=> mais sans hyperviseur !

LXC utilise namespaces, cgroups, et le noyau Linux de l’hôte.



Contenu d'un conteneur LXC

Il contient :

- Un système d'init (systemd)
- Plusieurs services
- Plusieurs utilisateurs
- Une arborescence Linux complète

Parfait pour l'approche Infra/système/devops/cyber.



Et LXD ??

C'est la couche de gestion d'un container LXC.
Avec on gère les images, les réseaux, le stockage, etc.



La conteneurisation applicative

Solution Docker :

- Isolation application/service
- Un processus principal
- Très léger car pas d'OS complet



L'orchestration de conteneurs (avancé)

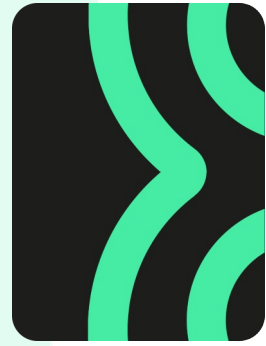
De base :

1 conteneur sur 1 machine pour 1 application/service

Si on augmente l'un des éléments la gestion se complique.

L'orchestration permet :

- Gestion de plusieurs conteneurs
- Répartition de charge
- Démarrage, arrêt (gestion du cycle de vie)
- Scalabilité horizontale et verticale (non-dynamique)



Introduction

*Le point de vue
développeur*

*Le point de vue
infrastructure*

Les solutions

Docker

Docker en pratique

Docker



Les concepts de Docker

L'idée générale de **Docker** est de **construire** (*build*) ou de **récupérer** (*pull*) des **images** dans un **dépôt** (*repository*) pour pouvoir les **exécuter** (*run*) sous la forme de **conteneurs** isolés les uns des autres ainsi que du système hôte.

Pour permettre aux conteneurs de communiquer, on peut s'appuyer sur des **volumes** qui permettent de partager des répertoires entre conteneurs (et avec l'hôte) ou des **réseaux** (virtuels) permettant aux conteneurs de communiquer entre eux.



Les images

Avec Docker, une **image** est l'ensemble des dossiers et fichiers qui constitue le système de fichier **racine** d'un **conteneur**, ainsi que les informations de configuration indiquant notamment quel programme doit être exécuté.

Une image peut être récupérée dans un **dépôt** (*repository*) ou **construite** (*build*) à l'aide d'un fichier de commandes : **Dockerfile**.



Les images (suite)

Les **images** Docker sont construites en **couches** (*layers*) chaque couche pouvant être conservée en cache et partagée entre différentes images (pour économiser de l'espace disque et du temps lors de la création d'images)

Une image équivaut à un programme avec toutes ses dépendances.



Les conteneurs → Exécuter du code avec Docker consiste à créer un **conteneur** à partir d'une **image**.

Un conteneur peut être vu comme une instance d'une image (Le conteneur est à l'image, ce que le processus est au programme).

Plusieurs conteneurs peuvent être créés à partir de la même image pour être exécutés simultanément ou non.

Les processus d'un conteneur ne peuvent accéder qu'aux fichiers de son image. Les modifications faites restent dans le conteneur et ne sont pas impactées sur l'image.



Les dépôts

Si **Docker** permet la construction **d'images** à partir d'un **Dockerfile**, il est courant pour partager ses **images** d'utiliser des **dépôts** (*repository*).

[DockerHub](#) est un service web géré par [Docker Inc.](#) qui héberge des dépôts publics et privés. Beaucoup d'applications courantes proposent sur Docker Hub des images utilisables directement (Serveurs web, BDD...).



Dépôt personnel

Il est aussi possible d'héberger son propre dépôt.

On peut alors **récupérer** (*pull*) ou **publier** (*push*) des **images** dans un **dépôt**.



Les volumes

→ Docker propose la création de **volumes** qui permettent de partager des fichiers entre le système hôte et les conteneurs.

L'idée générale des **volumes** est la création d'un **espace de stockage persistant, géré par docker** est pouvant être mis à disposition de conteneurs.

Un volume survit à la destruction d'un conteneur pour lui permettre le stockage de données persistantes.

Il est aussi possible de **monter** un dossier de l'hôte dans l'arborescence d'un conteneur. Ce mécanisme est appelé **bind mounts** par Docker.



Les réseaux → Docker permet, au lancement d'un conteneur, de **lier** un **port** du **système hôte** avec un **port** du **conteneur** et permet ainsi facilement l'utilisation de conteneurs pour exécuter des serveurs.

Par défaut les conteneurs ont une **interface réseau** sur un **pont** virtuel (*bridge*) géré par docker qui leur fournit une **configuration réseau** (via DHCP) et leur permet de sortir via un **NAT**.

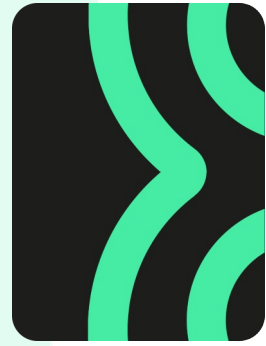
Docker permet aussi la création d'autres **réseaux virtuels** auxquels plusieurs conteneurs peuvent être connectés.



Considérations générales

Docker recommande qu'une image corresponde à un seul service (une application web, un serveur de base de données, un proxy...) et donc potentiellement de déployer plusieurs conteneurs pour mettre en place une application complète.

Un conteneur n'est pas censé durer dans le temps mais plutôt être redéployé fréquemment, notamment pour mettre à jour l'application.



Introduction

*Le point de vue
développeur*

*Le point de vue
infrastructure*

Les solutions

Docker

Docker en pratique

Docker en pratique



Les composants

Docker est constitué de plusieurs composants :

- **Docker Engine** :
 - Un Serveur de gestion des conteneurs (dockerd)
 - Une interface en ligne de commande (docker)
- **Docker Desktop** : Interface graphique - nécessaire sous windows
 - ⚠ Docker Desktop sous linux est une VM

Docker est disponible en version CE (Community Edition) ou via différents types de souscription.



Outils supplémentaires

Docker Compose :

Permet le lancement d'un ensemble de conteneurs (application multi-conteneurs) décrit par un fichier `docker-compose.yml`

Docker Swarm :

Regroupement de plusieurs Docker Engine en un cluster

Kubernetes :

Orchestration poussée de conteneurs



Docker CLI

L'utilisation de Docker en ligne de commande passe par la commande docker dont la syntaxe générale est :

`docker [OPTIONS] COMMAND`

Nécessite d'être dans le groupe **docker**.

```
wilder@host:~$ docker help
```

```
Usage: docker [OPTIONS] COMMAND
```

```
A self-sufficient runtime for containers
```

```
Options:  
[...]
```

```
Management Commands:  
[...]
```

```
Commands:  
[...]
```



Les images → La gestion des images se fait
via la commande :

`docker image COMMAND`

Par exemple :

`ls` : affiche les images dispo.

`pull` : récupère une image
dans un dépôt (DockerHub)

`rm` : supprime une image

Des alias, par ex. :

`docker images => docker
image ls`

```
wilder@host:~$ docker image --help
```

Usage: `docker image COMMAND`

Manage images

Commands:

<code>ls</code>	List images
<code>pull</code>	Pull an image or a repository from a registry
<code>push</code>	Push an image or a repository to a registry
<code>rm</code>	Remove one or more images
<code>tag</code>	Create a tag <code>TARGET_IMAGE</code> that refers to <code>SOURCE_IMAGE</code>



Lancer un conteneur

Création d'un conteneur :

```
docker run [OPTIONS]  
IMAGE [COMMAND] [ARG...]
```

Command remplace la
commande par défaut

Nombreuses options :
-p 8080:80 (port hôte 8080
-> 80)

```
wilder@host:~$ docker run --help
```

```
Usage: docker run [OPTIONS] IMAGE [COMMAND]  
[ARG...]
```

Run a command in a new container

Options:

-d, --detach	Run container in background and print container ID
-i, --interactive	Keep STDIN open even if not attached
-p, --publish list	Publish a container's port(s) to the host
-t, --tty	Allocate a pseudo-TTY



Gérer les conteneurs

Gestion des conteneurs :

`docker container COMMAND`

Par exemple :

`ls` : affiche les conteneurs

`start/stop` :

redémarrage/arrêt

De nombreux alias, par
exemple :

`docker ps => docker
container ls`

```
wilder@host:~$ docker container
```

Usage: `docker container COMMAND`

Manage containers

Commands:

<code>exec</code>	Run a command in a running container
<code>ls</code>	List containers
<code>rm</code>	Remove one or more containers
<code>start</code>	Start one or more stopped containers
<code>stop</code>	Stop one or more running containers
<code>[...]</code>	



Gérer les volumes

Gestion des volumes :

docker volume COMMAND

```
wilder@host:~$ docker volume --help
```

Usage: docker volume COMMAND

Manage volumes

Commands:

create	Create a volume
inspect	Display detailed information on one or more volumes
ls	List volumes
prune	Remove all unused local volumes
rm	Remove one or more volumes



Gérer les réseaux

Gestion des réseaux :

`docker network COMMAND`

```
wilder@host:~$ docker network --help
```

Usage: `docker network COMMAND`

Manage networks

Commands:

<code>connect</code>	Connect a container to a network
<code>create</code>	Create a network
<code>disconnect</code>	Disconnect a container from a
<code>network</code>	
<code>inspect</code>	Display detailed information on one
or	
<code>more networks</code>	
<code>ls</code>	List networks
<code>prune</code>	Remove all unused networks
<code>rm</code>	Remove one or more networks



Gestion globale

Gestion de docker :

`docker system COMMAND`

Par exemple :

`prune` : nettoyage des
éléments inutilisés

```
wilder@host:~$ docker system --help
```

```
Usage: docker system COMMAND
```

```
Manage Docker
```

```
Commands:
```

<code>df</code>	Show docker disk usage
<code>events</code>	Get real time events from the server
<code>info</code>	Display system-wide information
<code>prune</code>	Remove unused data



Le Dockerfile

Comment construire l'image ?

FROM : image de base (un seul FROM)

WORKDIR : cd dans le conteneur

COPY : Copie de fichiers locaux

RUN : exécute une commande

CMD : Commande exécutée par défaut au lancement (un seul CMD)

EXPOSE : ports en écoute

```
# Exemple d'une app node.js
```

```
FROM node:18-alpine
```

```
WORKDIR /app
```

```
COPY ..
```

```
RUN yarn install --production
```

```
CMD ["node", "src/index.js"]
```

```
EXPOSE 3000
```

[Dockerfile référence](#)



Générer une image

Création d'une image à partir
d'un dockerfile :

```
docker build [OPTIONS]  
PATH
```

```
wilder@host:~$ docker build --help
```

```
Usage: docker build [OPTIONS] PATH | URL | -
```

```
Build an image from a Dockerfile
```

```
-t, --tag list      Name and optionally a tag in the  
                    'name:tag' format
```



Cas d'usage

- Création de package de déploiement d'application
- Homogénéisation des environnements dev/test/pre-prod/prod
- Déploiements rapides et reproductibles
- Automatisation



Références

La [doc officielle](#)

La partie [Guides](#)

La partie [Manuals](#)

La partie [Reference](#)

Le [site Docker](#)

Notamment : [Docker 101 Tutorial](#)



Pour aller plus loin

[Guide de sécurisation ANSSI](#)

[Docker Compose](#)

[Docker Swarm](#)

[Kubernetes](#)

[Site d'Hadrien Pelissier](#)

[Site de Stéphane Robert](#)



Docker en résumé

À retenir

- Docker permet le déploiement d'application en conteneur.
- Il s'appuie sur les notions d'image, de conteneur, et de volume et permet la création de réseaux virtuels.
- Il utilise des fichiers dockerfile pour construire des images qui peuvent aussi être publiées dans des dépôts.
- Il constitue dans certains cas une alternative légère aux machines virtuelles.
- Peut simplifier la relation entre les équipes dev et ops.



En résumé

À retenir

- La conteneurisation isole les applications au niveau du système
- Conteneurs et machines virtuelles répondent à des besoins différents
- Docker structure le déploiement applicatif
- L'orchestration permet le passage à l'échelle

La conteneurisation est une réponse moderne aux enjeux de déploiement, de sécurité et de scalabilité des applications dans les infrastructures actuelles.



MERCI

pour votre
participation.

C'est à vous
maintenant.
Des questions ?
Des remarques ?

